

PROJECT DOCUMENTATION

AI-Powered Banking Customer Support Agent

Full-Stack Banking Platform with a Conversational AI Assistant

React • Spring Boot • MySQL • FastAPI • RAG • LLM Tool-Calling

Consolidated Technical Reference

Architecture • Workflows • Technology Stack • Setup Guide

1. Project Overview

This project is a **full banking application** (like a mini online bank) that also has an **AI chatbot** built into it. Customers can either click around the app normally, or simply type a message to the AI assistant in English, Hindi, or a mix of both, and it will do the work for them.

The AI assistant is not just a chatbot that talks — it is actually connected to the real banking system. So when a user says “transfer 500 rupees to my friend,” the AI checks that this is safe and allowed, and then performs the real transfer using the banking APIs. In simple words: it understands language using an LLM (a smart language model), looks up answers using RAG (a smart search over stored knowledge), and takes real actions using tool-calling (calling the right function for the right job).

Objectives

In short, the goals of this project are: understand what the user is asking in plain language, carry out banking tasks safely, reduce the amount of work human support staff need to do, guide users when they're confused, and make banking feel faster and easier overall.

Project Goal

The main goal is simple: build a banking assistant that can safely do real banking tasks, answer questions, and talk to users the way a helpful human support agent would — by combining LLMs, RAG, and the banking backend together.

2. Technology Stack

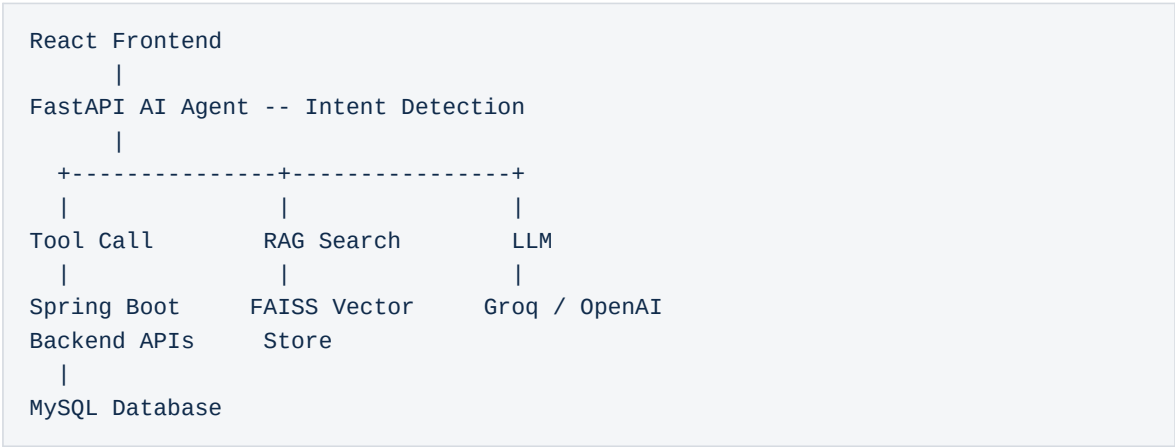
Layer	Technologies	Core Responsibilities
Frontend	React.js 18, React Router DOM, Axios, HTML5, CSS3, JavaScript (ES6+)	Authentication UI, dashboard, deposit / withdraw / transfer, transaction history, profile management, admin dashboard, AI chatbot UI
Banking Backend	Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA, Hibernate, JWT Authentication, REST APIs, Maven	User authentication, customer management, transaction processing, ticket management, business logic, database operations, API security
Database	MySQL 8.x	Core tables: customers, transactions, tickets, support_agents, ticket_messages
AI Agent	Python 3.11+, FastAPI, Uvicorn, Requests, Sentence Transformers, FAISS, Groq API (Llama 3.1 8B Instant), OpenAI (optional fallback), LangChain-ready	Intent detection, tool calling, RAG search, session management, context memory, fraud detection, conversational AI

AI Concepts Applied — in Plain Words

LLM (Large Language Model)	The “brain” that reads and writes natural language.
RAG (Retrieval-Augmented Gen.)	Looks up answers from stored notes before replying.
Tool Calling	Lets the AI actually click the button for you, in code.
Intent Detection	Figures out what the user really wants to do.
Context Memory	Remembers earlier parts of the same conversation.
Session Management	Keeps track of who is chatting and for how long.
Fraud Detection	Flags suspicious or risky transactions.
API Orchestration	Coordinates calls between different services smoothly.

3. System Architecture

Think of this project as three separate helpers that talk to each other: the **website** (what the user sees), the **banking backend** (the real money and account logic), and the **AI agent** (the smart assistant). Whenever a user sends a chat message, the AI agent reads it and decides one of three things: “should I run a banking action?”, “should I look this up in my notes?”, or “should I just generate a helpful reply?”



4. Application & AI Workflows

4.1 What Happens on the Website

The user logs in and gets a JWT token (basically a digital ID card that proves who they are). From the dashboard, they can deposit, withdraw, transfer money, check their transactions, update their profile, or open the AI chat — every click sends a secure request to the backend, and the answer is shown back on screen. Only logged-in customers can see the chatbot; it's hidden from the login page and from the admin side.

4.2 What Happens on the Backend

The backend is the part that actually manages money and accounts, split into five areas: Login, Customers, Transactions, Support Tickets, and Admin. Before doing anything, it always double-checks: is the login token valid, is this person allowed to do this, is the data correct, and does it follow the bank's rules? Only after all checks pass does it touch the database.

4.3 How the AI Agent Thinks

Every time a user sends a message, the AI follows the same simple steps:

- 1 Check if there's an ongoing conversation and recall what was said earlier.
- 2 Figure out what the user is trying to do (this is "intent detection").
- 3 **If it's a banking task** (like "check my balance"): run the matching tool, which calls the real backend API and brings back the result.
- 4 **If it's not a banking task** (like a general question): first search the stored knowledge notes; if nothing useful is found, ask the LLM to write a normal, helpful reply.
- 5 Send the final answer back to the website so the user can see it.

4.4 The Whole Journey in One Line

Login → secure request → backend checks it → user's message goes to the AI → AI runs a tool / searches notes / writes a reply → answer shows up on the user's screen.

5. AI Capabilities & Knowledge Base

5.1 Supported AI Capabilities

- | | |
|---------------------------------------|---|
| ✓ Balance Inquiry | ✓ Banking FAQs & Process Guidance |
| ✓ Transaction History & Filtering | ✓ Session Memory & Context Awareness |
| ✓ Money Transfer | ✓ English, Hindi & Mixed-Language Queries |
| ✓ Support Ticket Creation | ✓ Tool Calling / REST API Integration |
| ✓ Fraud Detection | ✓ RAG-Based Responses |
| ✓ High-Value Transaction Confirmation | ✓ Multi-Step Conversations |

5.2 RAG — How the AI “Looks Things Up”

Imagine a small notebook, `rag/data.txt`, where all the banking know-how is written down in plain text — how to deposit, withdraw, transfer, update a profile, check history, or raise a ticket, plus common FAQs and rules. This notebook is turned into “searchable numbers” (called embeddings) and stored in a fast search tool called FAISS. So when a user asks a question, the AI quickly searches this notebook for the closest matching answer instead of guessing. **Whenever someone updates the notebook, this search index has to be rebuilt so the AI learns the new information.**

6. Security Features

- JWT-based authentication on every protected API
- Duplicate request prevention
- High-value transaction confirmation
- Self-transfer prevention

- Insufficient balance validation
- Input validation on all endpoints
- Session timeout / session management

7. Folder Structure

Frontend	Banking Backend	AI Agent
components/ pages/ layout/ context/ utils/ styles/ services/ assets/	controller/ service/ service/impl/ repository/ entity/ dto/ security/ config/ util/ resources/	main.py, agent.py llm.py, tools.py tool_executor.py memory_store.py ticket.py, escalation.py config.py rag/ (data.txt, rag_engine.py, build_rag.py, vector_store.pkl)

Key AI-Agent Modules

File	Purpose
agent.py	Core AI workflow — intent detection, routing, session management, fraud logic, transfer confirmation, ticket flow
tool_executor.py	Executes banking tools: balance, transactions, transfer, complaint handling
tools.py	Wraps and calls the Spring Boot REST APIs
rag_engine.py	Builds/loads the FAISS index and performs semantic search
llm.py	Communicates with Groq / OpenAI and returns structured tool calls
main.py	FastAPI entry point — receives chatbot requests and returns AI responses

8. Project Setup Guide

1. Clone the repositories

Clone the frontend, backend, and AI-agent source repositories.

2. Start MySQL

Create the required database and import the SQL schema if needed. Update credentials in application.properties.

3. Start the Spring Boot backend

Requires Java 17 and Maven.
Run: mvn spring-boot:run
Default URL: http://localhost:8001

4. Start the AI agent

Requires Python 3.11+.
Install dependencies: pip install -r requirements.txt

If rag/data.txt was modified, delete rag/vector_store.pkl and rebuild with python -m rag.build_rag.
Start the server: python main.py
Default URL: http://localhost:8002

5. Start the React frontend

npm install
npm run dev
Default URL: http://localhost:5173

9. Limitations & Future Enhancements

9.1 Current Limitations

- Voice support only works as well as the user's web browser allows.
- If someone edits the knowledge notebook (data.txt), the search index has to be rebuilt by hand.
- Chat sessions are stored in the computer's memory for now; moving to Redis would make this more reliable for a real production system.
- Fraud detection currently just follows fixed rules; it could later be upgraded to a smarter ML model.

9.2 Future Enhancements

Voice-to-Voice / Speech-to-Speech Conversation	Redis Session Store
Real-Time Speech Recognition	Live / Streaming AI Responses
OCR-Based Document Upload	Personalized Financial Insights
Investment Recommendations	Multi-Bank Integration
Live Customer-Support Handover	Advanced Analytics Dashboard
Email Notifications	Production Deployment: Docker, Kubernetes, CI/CD

10. Resume Description

Two versions are given below — pick whichever fits the space you have on your resume.

10.1 Short Version (1 line)

Built a full-stack banking app with an AI chatbot (React, Spring Boot, MySQL, FastAPI) that lets users check balance, transfer money, and get support using natural language, powered by RAG and LLM tool-calling.

10.2 Medium Version (3–4 lines / bullet points)

- Developed a full-stack banking platform (React, Spring Boot, MySQL) with a conversational AI support agent (Python, FastAPI) that understands English, Hindi, and mixed-language queries.

- Built an AI agent that performs real banking actions (balance check, fund transfer, ticket creation) through secure REST APIs using tool-calling, instead of just chatting.
- Implemented a RAG-based knowledge search (Sentence Transformers + FAISS) so the AI can answer banking FAQs and process questions accurately from a custom knowledge base.
- Added security features such as JWT authentication, fraud-detection rules, duplicate-request prevention, and transaction validation to keep the system safe.

11. Interview Explanation

Use this as a simple, spoken-style answer for “Tell me about this project” in an interview. Say it in your own words — this is just the structure to follow.

11.1 "What is it?"

“It’s a banking application where users can do normal things like deposit, withdraw, transfer money, and check their transaction history. On top of that, I added an AI chatbot that lets users do the same things just by typing a message — in English, Hindi, or even a mix of both.”

11.2 "How does the AI actually work?"

“When a user sends a message, my AI agent first figures out what they want — this is called intent detection. If it’s a banking task, like checking balance or transferring money, the AI calls the actual backend API and performs the action for real, after checks like login and fraud detection pass. If it’s more of a question, like ‘how do I update my profile’, the AI first searches a small knowledge base I built using RAG. If it still can’t find a good answer, it asks an LLM (Groq’s Llama model) to generate a normal, human-like reply.”

11.3 "Why did you use RAG instead of just the LLM?"

“Because an LLM on its own can guess or make mistakes about our specific banking rules. RAG lets me store our own accurate information — our processes, our FAQs, our policies — and the AI searches that first, so answers stay accurate and specific to our bank instead of being generic.”

11.4 "How did you keep it secure?"

“Every request needs a valid JWT token to prove the user is logged in. I also added checks like confirming high-value transfers, blocking transfers to yourself, checking for insufficient balance, and basic fraud-detection rules — so the AI can’t be tricked into doing something harmful.”

11.5 "What was challenging?"

“Making sure the AI didn’t just talk, but actually performed safe, correct banking actions was the hardest part — that meant carefully connecting the AI agent to the real backend APIs with proper validation, and testing it in English, Hindi, and mixed-language messages so it understood users properly.”